

# Теория параллелизма

## Отчёт Задача №8

Сейгель Арина Сергеевна, 23932

г. Новосибирск

2025 г.

Цель: попробовать на практике использование cudaAPI на примере решения уравнения теплопроводности разностной схемой, протестировать время работы с использованием cuda ядер и cuda graph.

Компилятор: g++ Профилировщик:

Nsight Systems

Замеры времени: std::chrono::high\_resolution\_clock (итоговое время в секундах)

## Выполнение на CPU

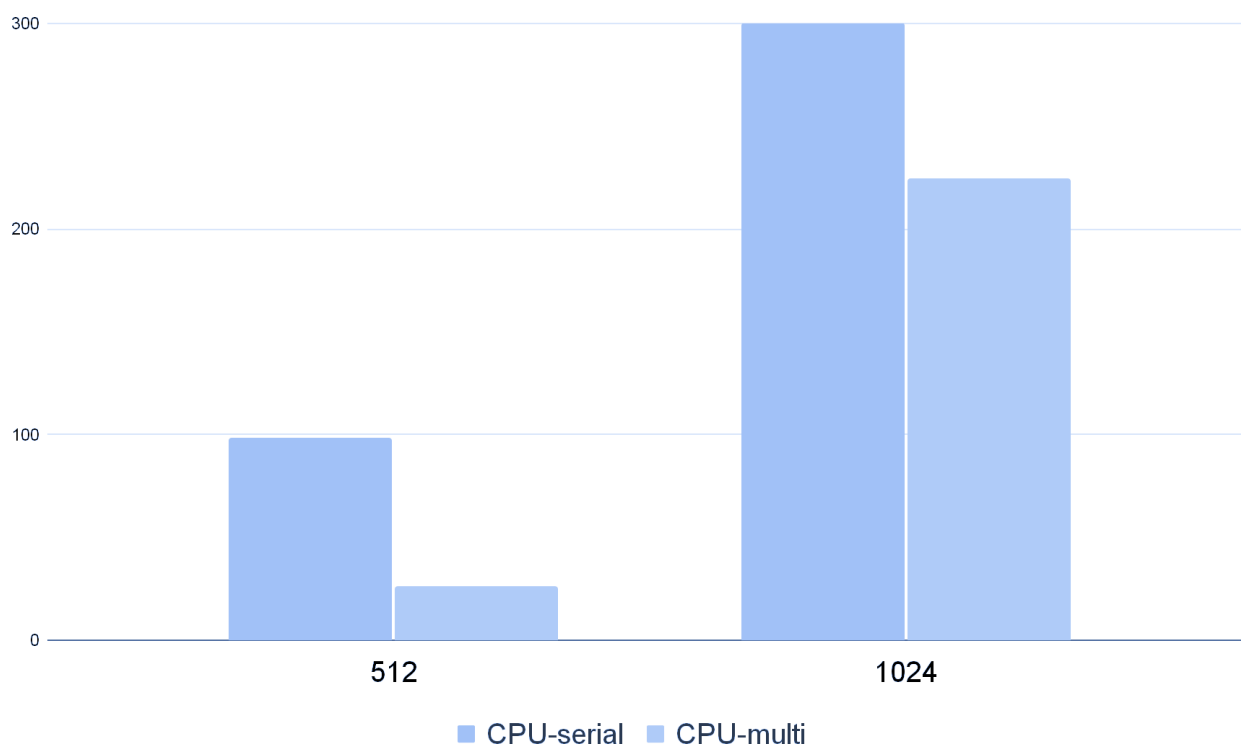
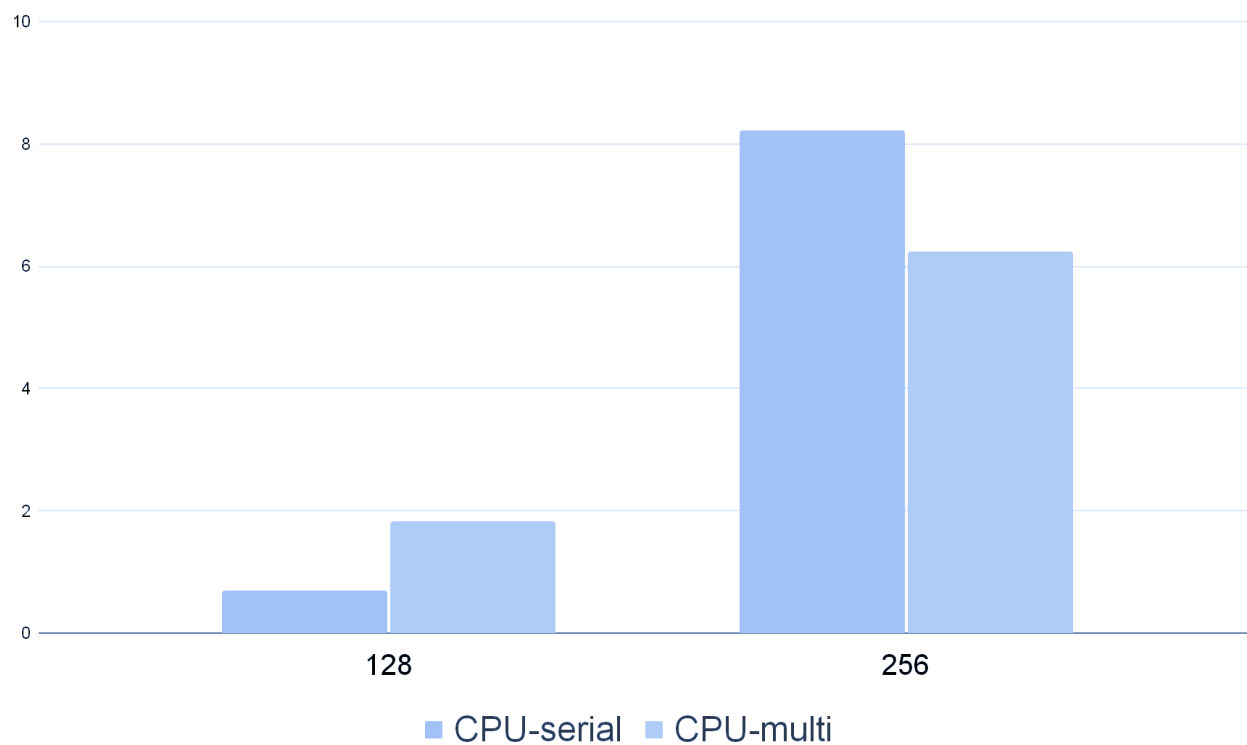
### CPU-onescore

| Размер сетки | Время<br>выполнения | Ошибка      | Количество<br>итераций |
|--------------|---------------------|-------------|------------------------|
| 128*128      | 0.703 сек.          | 4.79656e-08 | 40000                  |
| 256*256      | 8.229 сек.          | 5.82721e-07 | 110000                 |
| 512*512      | 98.256 сек.         | 9.92435e-07 | 340000                 |
| 1024*1024    | >25 мин.            | -           | 1000000                |

### CPU-multicore

| Размер сетки | Время<br>выполнения | Ошибка      | Количество<br>итераций |
|--------------|---------------------|-------------|------------------------|
| 128*128      | 1.84 сек.           | 4.79656e-08 | 40000                  |
| 256*256      | 6.244 сек.          | 5.82721e-07 | 110000                 |
| 512*512      | 26.077 сек.         | 9.92435e-07 | 340000                 |
| 1024*1024    | 224.662 сек.        | 1.36929e-06 | 1000000                |

Диаграмма сравнения CPU-serial и CPU-multi

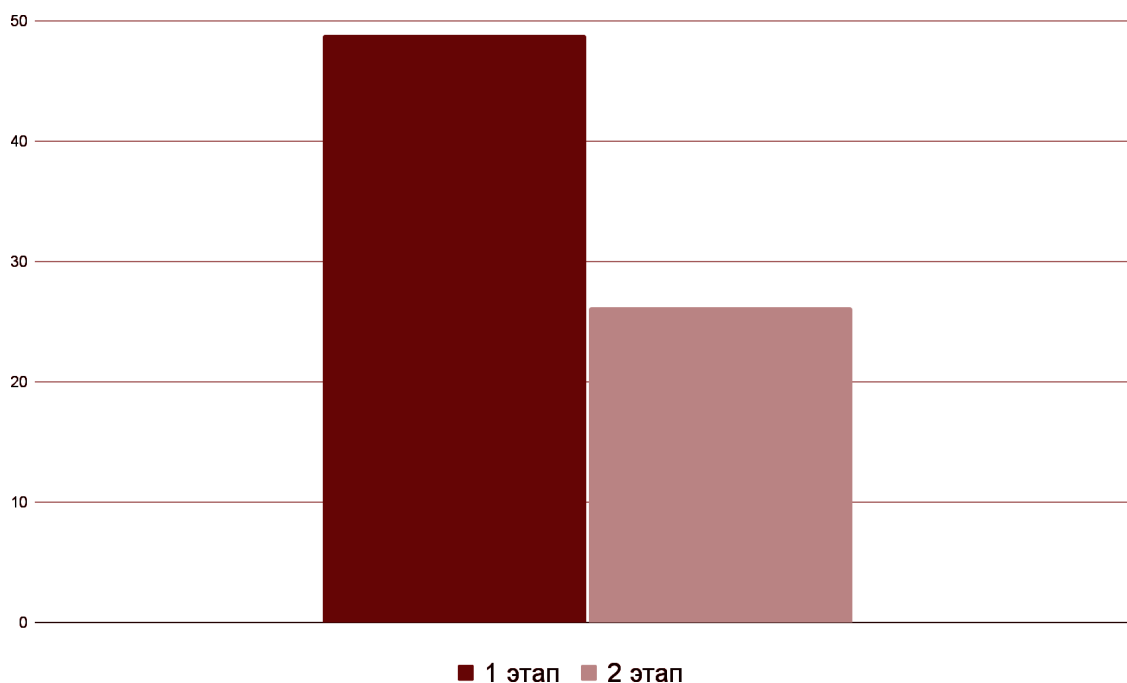


## Выполнение на GPU

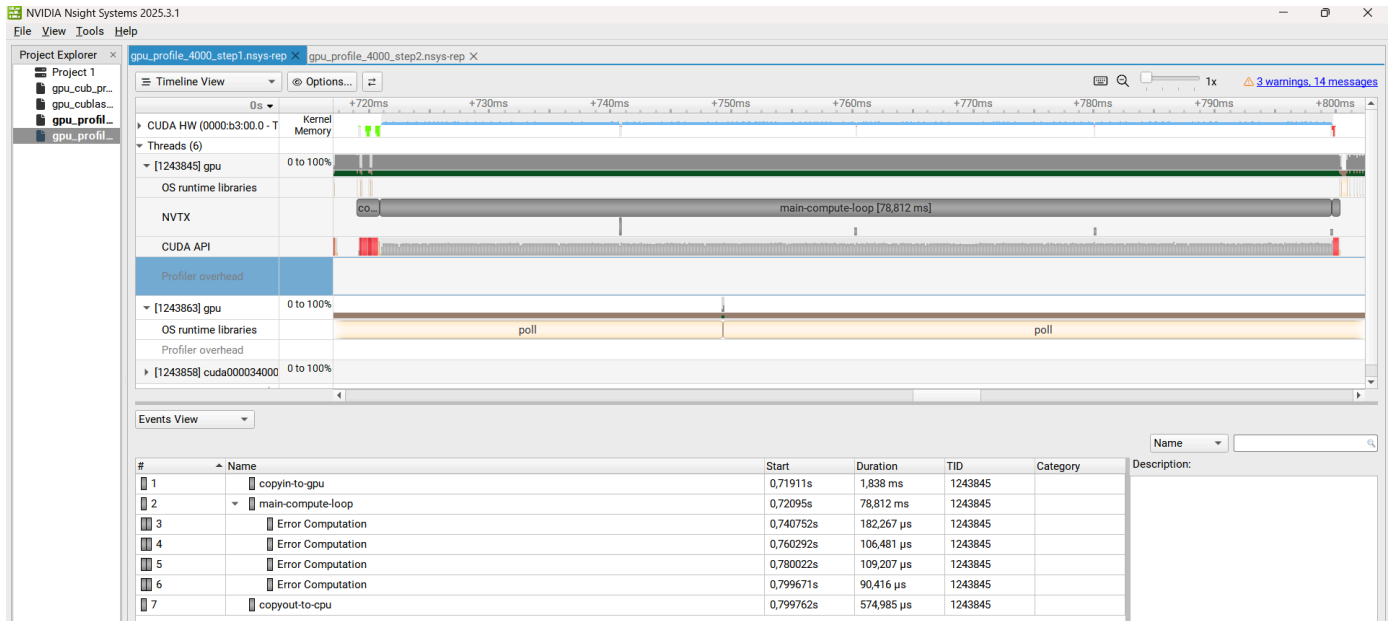
### Этапы оптимизации на сетке 512\*512

| Этап № | Время выполнения | Ошибка      | Максимальное количество итераций | Комментарии                                       |
|--------|------------------|-------------|----------------------------------|---------------------------------------------------|
| 1      | 48.8017 сек.     | 1.36929e-06 | 1000000                          | Базовое применение openACC для ускорения расчётов |
| 2      | 26.1976 сек.     | 1.36929e-06 | 1000000                          | Использование async при использовании openACC     |

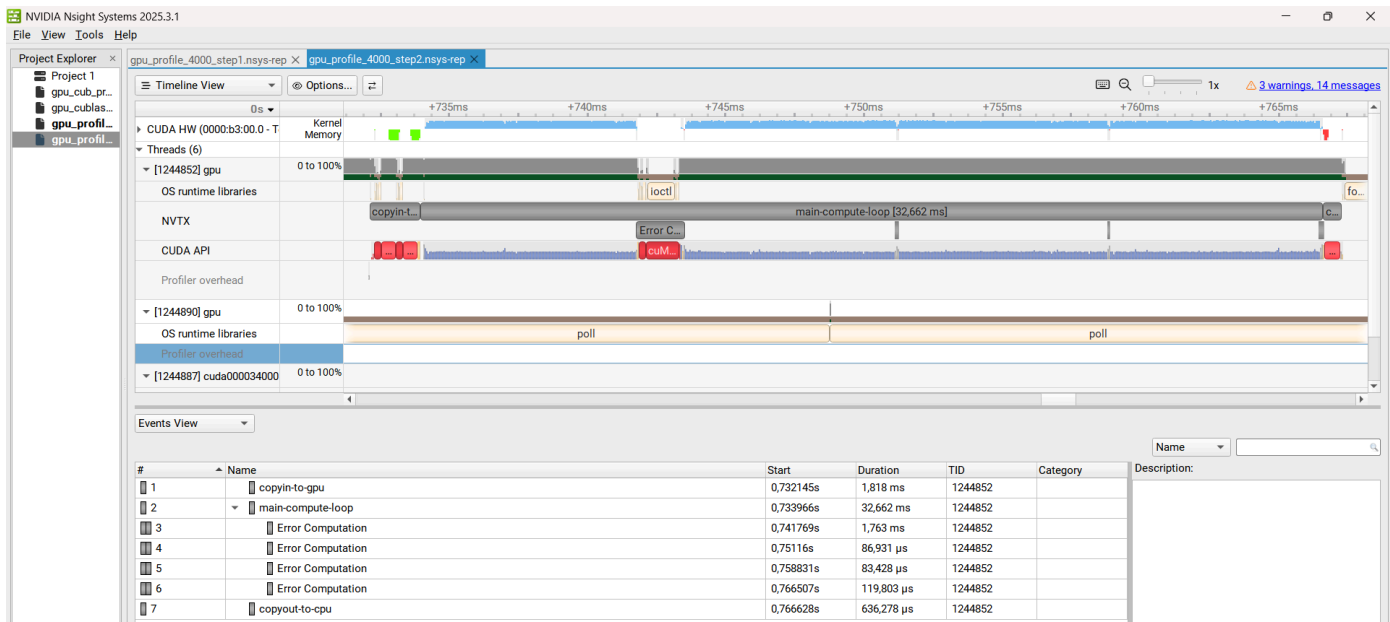
### Диаграмма оптимизации



## Профилрование этап 1



## Профилрование этап 2



## GPU - оптимизированный вариант

| Размер сетки | Время выполнения | Точность    | Количество итераций |
|--------------|------------------|-------------|---------------------|
| 128*128      | 0.110 сек.       | 4.79656e-08 | 40000               |

|           |             |             |         |
|-----------|-------------|-------------|---------|
| 256*256   | 0.345 сек.  | 5.82721e-07 | 110000  |
| 512*512   | 1.785 сек.  | 9.92435e-07 | 340000  |
| 1024*1024 | 25.431 сек. | 1.36929e-06 | 1000000 |

### Вывод матрицы 10\*10

Размер=10

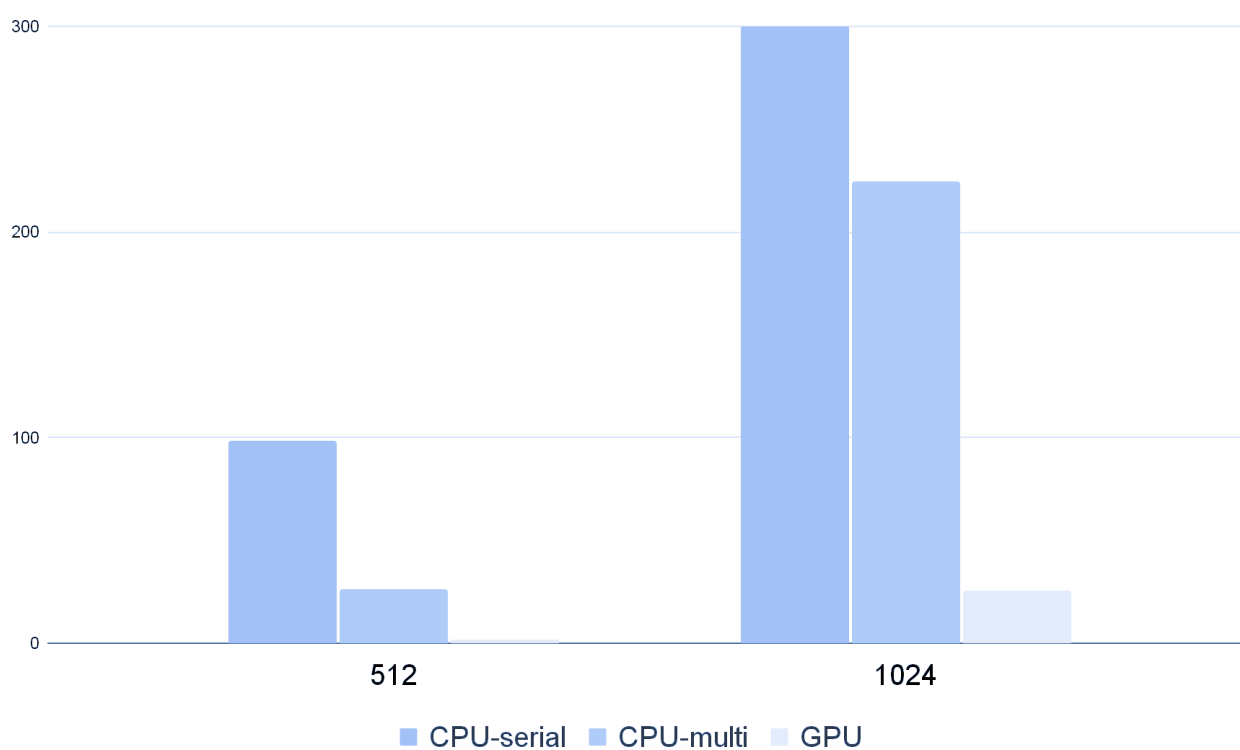
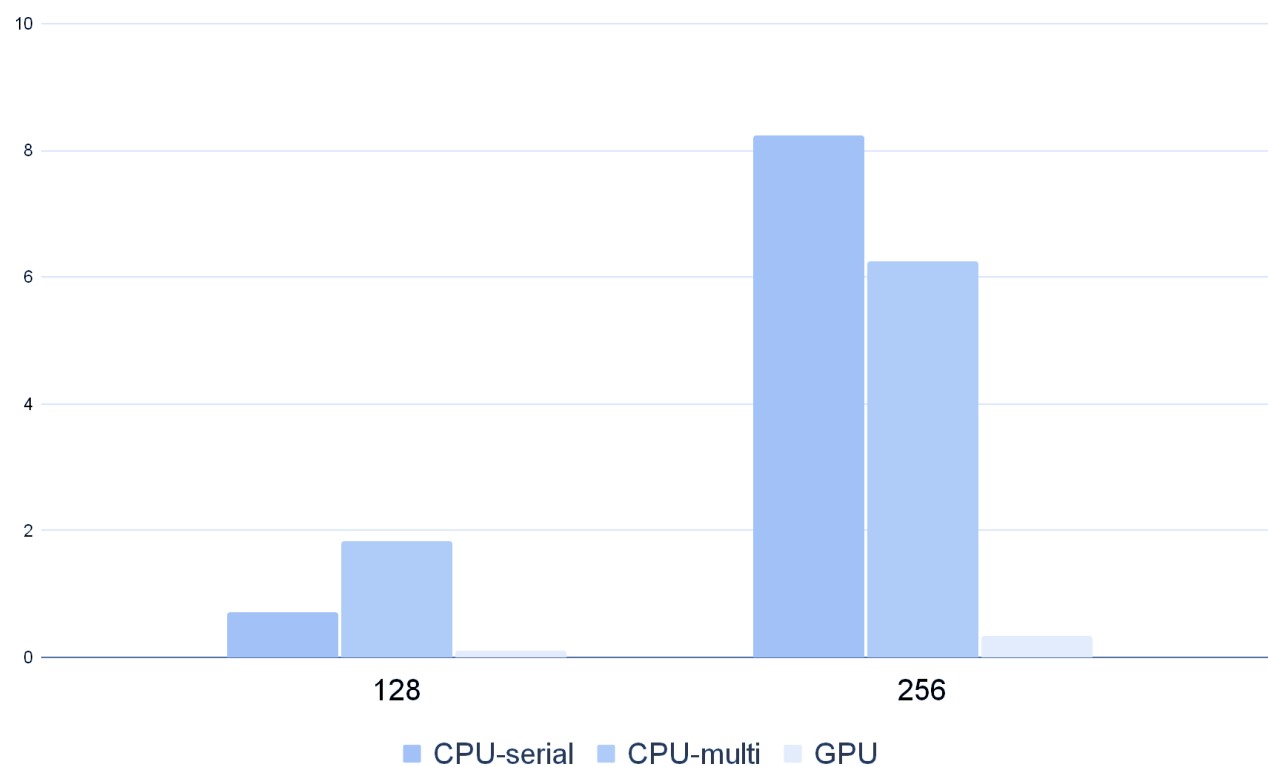
Время: 0.0284797 сек, Ошибка: 0, Итерации: 10000

```

10 11.1111 12.2222 13.3333 14.4444 15.5556 16.6667 17.7778 18.8889 20
11.1111 12.2222 13.3333 14.4444 15.5556 16.6667 17.7778 18.8889 20 21.1111
12.2222 13.3333 14.4444 15.5556 16.6667 17.7778 18.8889 20 21.1111 22.2222
13.3333 14.4444 15.5556 16.6667 17.7778 18.8889 20 21.1111 22.2222 23.3333
14.4444 15.5556 16.6667 17.7778 18.8889 20 21.1111 22.2222 23.3333 24.4444
15.5556 16.6667 17.7778 18.8889 20 21.1111 22.2222 23.3333 24.4444 25.5556
16.6667 17.7778 18.8889 20 21.1111 22.2222 23.3333 24.4444 25.5556 26.6667
17.7778 18.8889 20 21.1111 22.2222 23.3333 24.4444 25.5556 26.6667 27.7778
18.8889 20 21.1111 22.2222 23.3333 24.4444 25.5556 26.6667 27.7778 28.8889
20 21.1111 22.2222 23.3333 24.4444 25.5556 26.6667 27.7778 28.8889 30

```

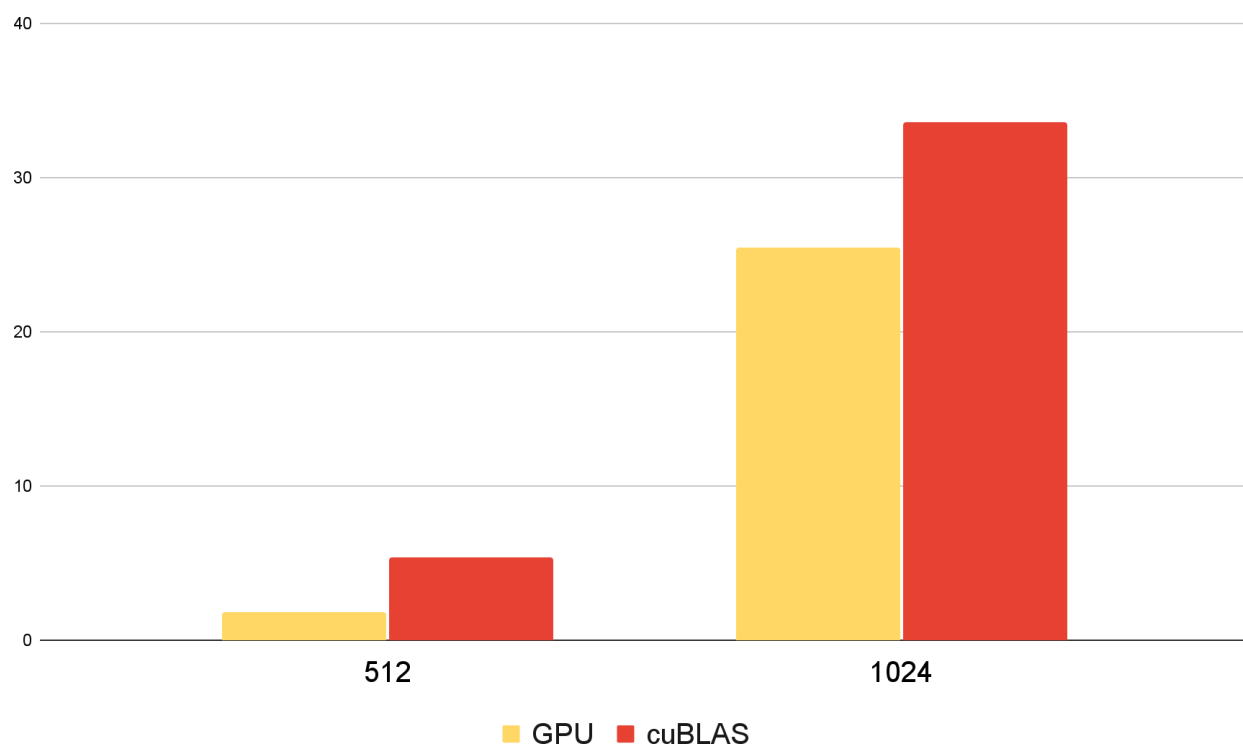
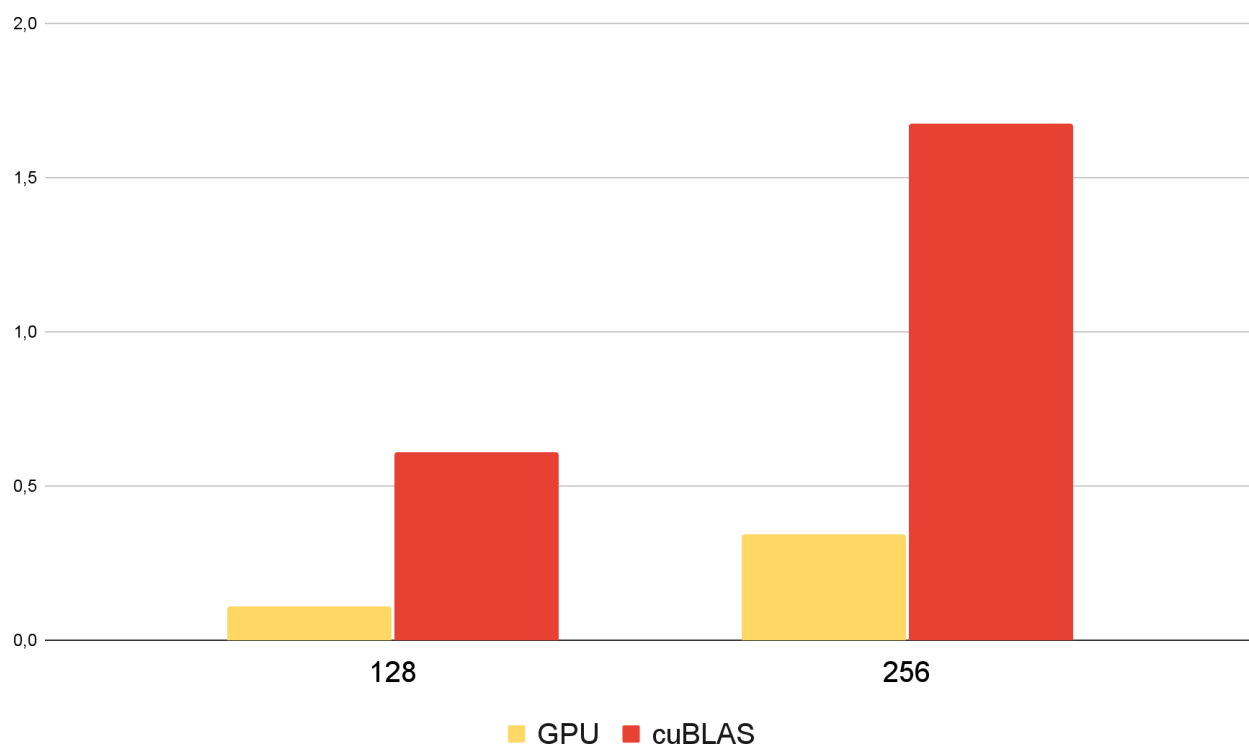
Диаграмма сравнения времени работы CPU-serial, CPU-multi, GPU(оптимизированный вариант) для разных размеров сеток





## Замена редукции на функции библиотеки cuBLAS

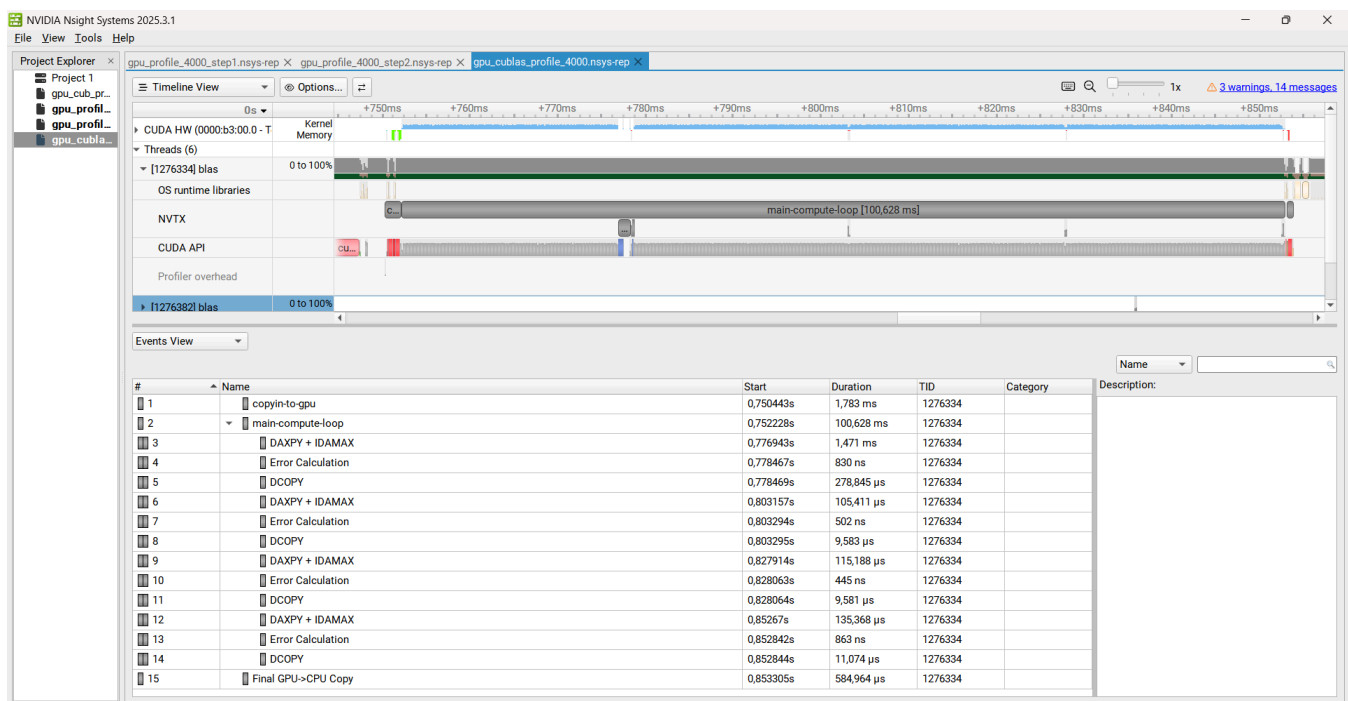
### Сравнение программ с cuBLAS и без



## cuBLAS реализация

| Размер сетки | Время<br>выполнения | Ошибка      | Количество<br>итераций |
|--------------|---------------------|-------------|------------------------|
| 128*128      | 0.611 сек.          | 4.79656e-08 | 40000                  |
| 256*256      | 1.677 сек.          | 5.82721e-07 | 110000                 |
| 512*512      | 5.394 сек.          | 9.92435e-07 | 340000                 |
| 1024*1024    | 33.620 сек.         | 1.36929e-06 | 1000000                |

## Профилирование

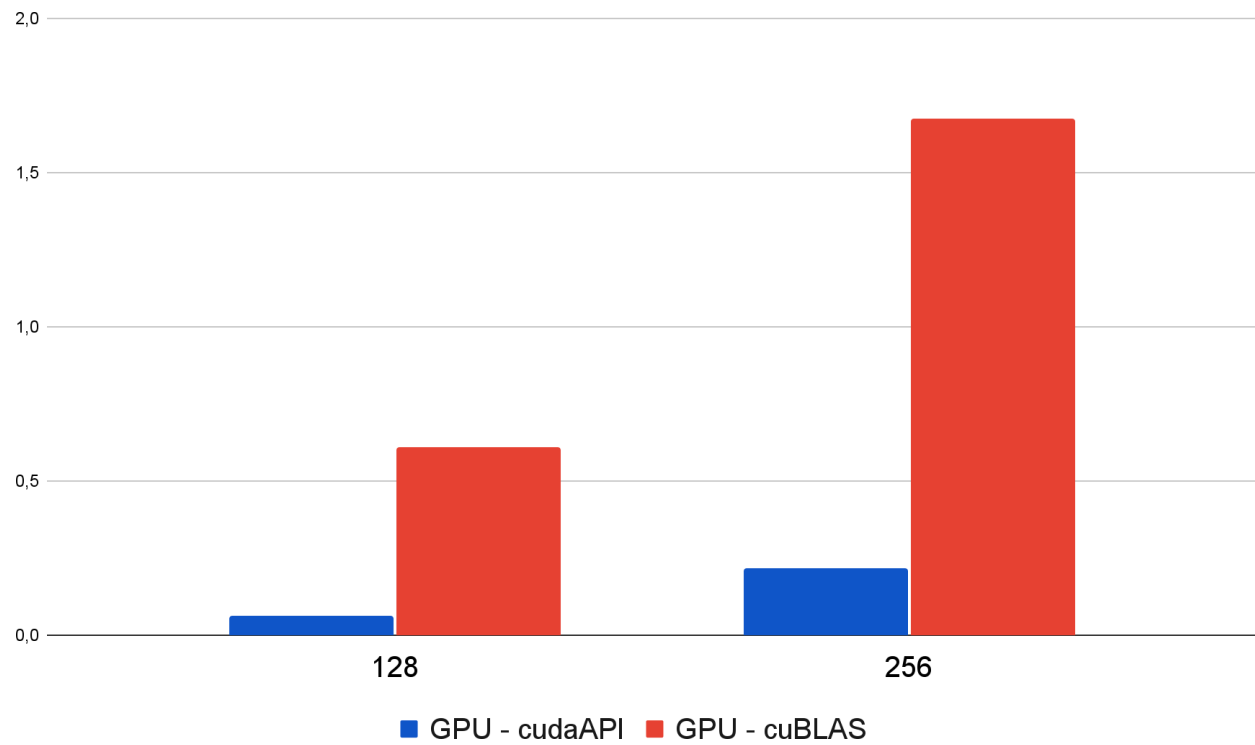


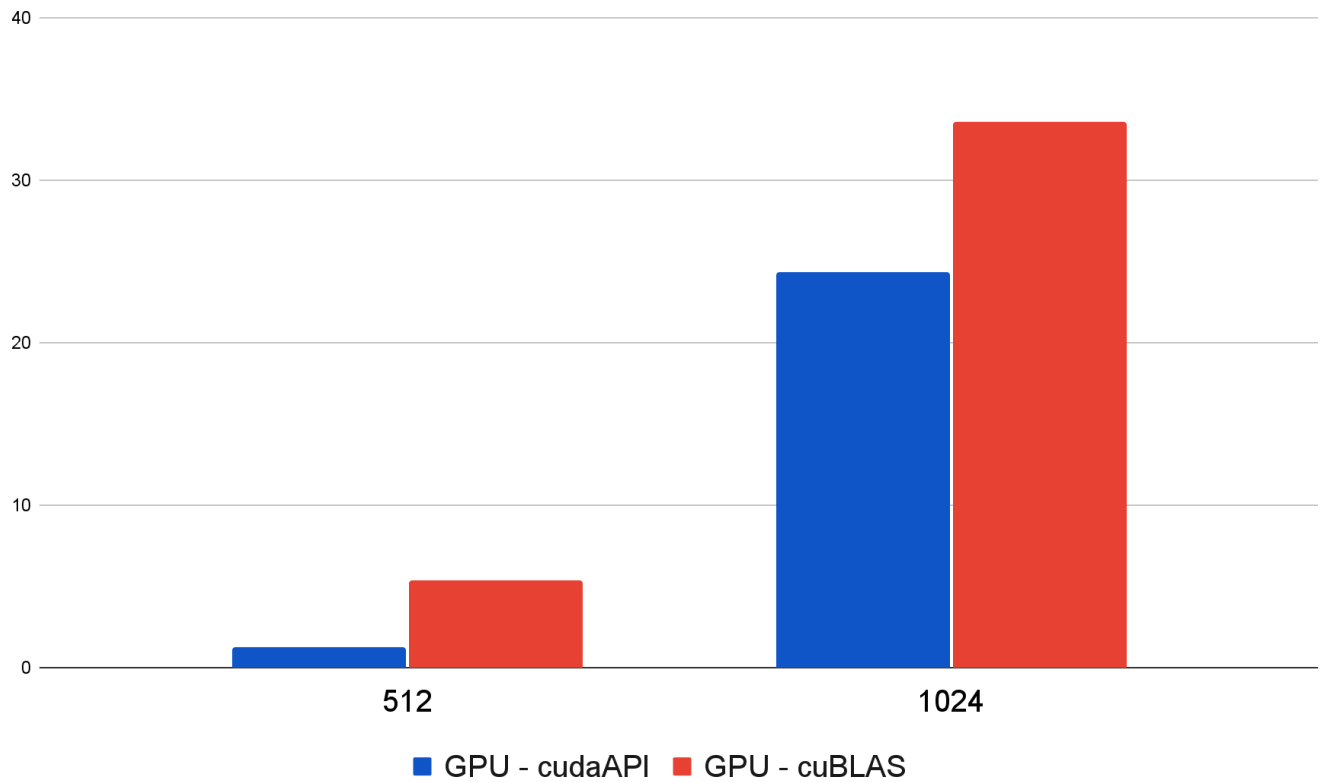
## GPU - cudaAPI

| Размер сетки | Время<br>выполнения | Ошибка      | Количество<br>итераций |
|--------------|---------------------|-------------|------------------------|
| 128*128      | 0.065 сек.          | 7.53244e-07 | 31000                  |
| 256*256      | 0.216 сек.          | 9.91241e-07 | 103000                 |

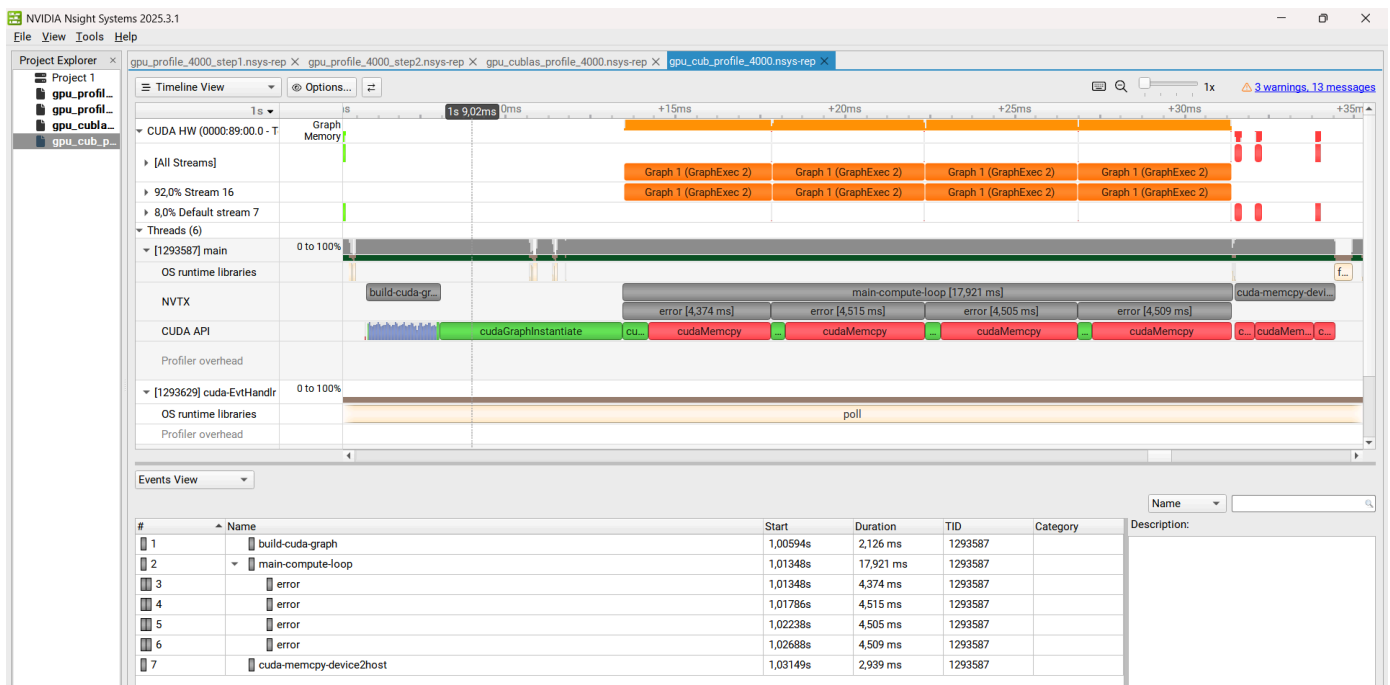
|           |             |             |         |
|-----------|-------------|-------------|---------|
| 512*512   | 1.302 сек.  | 9.92435e-07 | 340000  |
| 1024*1024 | 24.320 сек. | 1.36929e-06 | 1000000 |

### Сравнение программ с cuBLAS и cudaAPI

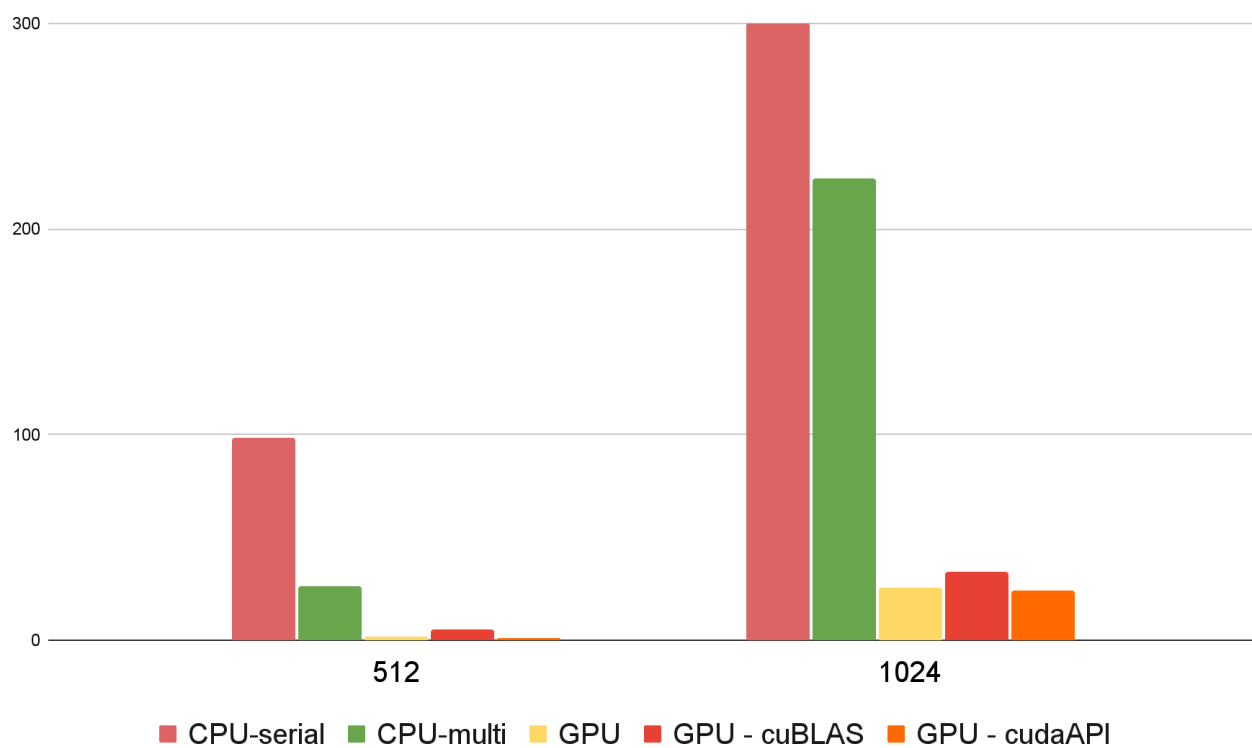
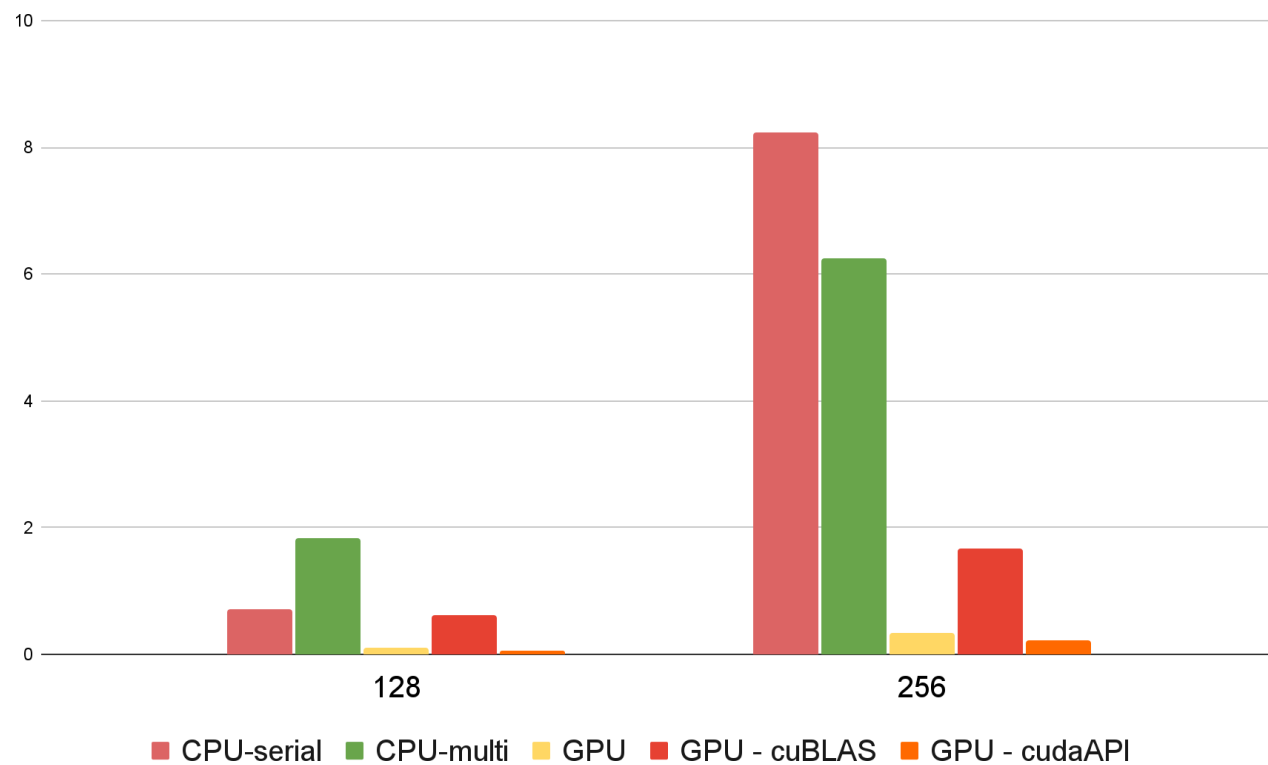




## Профилирование



## Сравнение всех вариантов реализаций



## Вывод

Задействование нескольких ядер CPU заметно повышает производительность, однако на любом размере матрицы наилучшие результаты демонстрирует GPU. Разница между реализациями, в которых ошибка вычисляется с помощью директив OpenACC или cuBLAS, невелика. Наивысшую производительность обеспечивают вычисления с использованием CUDA, превосходя даже обычный GPU-вариант.